1 研究背景与目前系统现状

1.1 研究背景

对于蛋白质设计这类高代价任务而言，系统真正面对的难点不只是”是否会调用工具“，而在于以下几个方面：

- 任务链条长，且阶段之间存在强依赖关系；
- 不同阶段的计算代价和失败后果高度不对称；
- 运行过程中常常出现信息不充分、候选分歧、局部失败和恢复决策问题；
- 何时继续自动执行、何时切换策略、何时请求人工介入，本身就是关键问题。

因此，如果将毕设继续定义为“蛋白质设计助手”，其研究重点很容易停留在“任务分解、工具调用、结果输出”这一层面，即便加入了风险门控等系统。如果希望体现更明确的计算机相关的专业特征的话，我觉得可以把问题进一步上升为“高代价科学工作流的执行控制与治理”。

1.2 当前系统现状

从现有的系统设计来看，它已经形成了一个较为稳定的多 Agent 工作流执行系统。

首先，系统采用了显式有限状态机（FSM）作为运行核心，要求任务状态显式、状态转移合法且可记录，外部语义状态和内部执行状态均有明确定义。这意味着系统已经具备较好的执行一致性、可恢复性与可审计性基础。

其次，系统在角色划分上已经形成了较为清晰的职责边界：PlannerAgent 负责生成计划候选，ExecutorAgent 负责工具执行和恢复流程，SafetyAgent 负责评估，SummarizerAgent 负责结果汇总。这种结构表明系统已经具备典型多 Agent runtime 的雏形。

再次，系统已经围绕蛋白质设计任务构建了较为完整的阶段化流程，已经支持：

- S1 序列候选生成
- S2 结构投影
- S3 质量门控
- S4 基于结构条件的细化
- 分层 patch/replan 恢复机制

此外，系统已经具备 HITL（Human-in-the-loop）机制和恢复链设计，包括 `WAITING_*` 以及 $retry \to patch \to replan$ 的恢复顺序要求。这说明系统已经在工程层面充分考虑了可恢复执行和人工参与的问题。

最后，系统已经具备一定实验与评测基础。当前仓库中存在 A0-A6 纵向实验框架，用于从效果、机制、代价、治理维度评估系统表现，并支持复现与增量归因分析。这为后续研究提供了较好的实验基础。

1.3 对当前系统的整体评估

综合来看，当前系统已经具备：

- 较成熟的多 Agent runtime 骨架；
- 显式状态机与和恢复执行机制；
- 蛋白质设计的具体工作流场景；
- 实验框架和评测口径

但从毕设角度考虑，该系统也存在一个更关键的问题：其更像是一个完成度较高的执行平台，而不是一个核心算法问题已经被明确抽象出来的研究课题。

这一问题主要体现在三个方面。

第一，当前系统虽然已经具备打分、风险等级、代价评估等概念，但从实现上来看，这些更多是启发式聚合结果，主要服务于候选排序和展示，尚未形成一个足够独立、足够可论证的核心算法。

第二，系统当前的主要研究对象仍然是“生命科学任务 → 多 Agent 规划执行 → 工具调用 → 结果输出”。这意味着，即便加入了门控、人工决策、patch/replan 等机制，它在选题层面仍然会被看成“生命科学助手”的一部分或一部分的增强版，而非一个独立成立的计算机算法课题。

第三，当前系统的许多优点更多体现在“工程正确性”和“执行健壮性”上面。例如状态一致性、审计、恢复、人工确认等。这些无法突出算法上的创新，因此必须进一步回答：系统究竟依据什么做出关键治理决策，如何量化任务难度、执行风险和工作量，并据此改变执行路径。

小结 因此，对当前系统的总体判断可以概括为：当前系统已经具备成为实验平台和实验载体的条件，但是尚未形成一个足够清晰、足够独立的核心算法主线。它最大的风险是“研究对象还没有从生命科学助手（或蛋白质设计平台）提升为更底层、更通用的计算机问题”。如果不进行这一提升，那么即便继续为系统添加功能，也仍然可能被视为另一生命科学多 Agent 助手系统的补充或子集，而难以形成一个边界清晰、创新点明确的毕业设计选题。

2 当前选题面临的核心问题

在已有系统的基础上，当前毕设如果继续按原有思路推荐，其主要问题不在于“系统功能不够丰富”，而在于选题主线尚未完成从应用系统到研究问题的提升。这使得当前选题在创新性、独立性和答辩的可解释性方面都存在明显风险。

1. 缺少一个足够明确的核心算法问题 目前系统已经包含多 Agent 协作、显式状态机、HITL、`patch/replan`、蛋白质设计工作流等多个组成部分，但这些组成部分更多体现为一个较完整的执行平台，而不是一个被单独抽象和证明的算法对象。

换句话说，当前系统解决了“如何把流程跑起来”的问题，却没有清晰回答“系统依据什么原则做关键决策”这一更为核心的问题。尤其在以下方面，仍然缺少关键建模：

- 如何刻画任务难度
- 如何估计执行风险
- 如何衡量工作量与代价
- 如何在不同风险和预算条件下调整执行策略
- 如何决定是否值得自动执行、请求人工接入或直接恢复/终止

虽然当前已有部分门控和打分机制，但更多属于启发式聚合和排序，而不是一个足够独立、可被论证为“核心算法”的研究对象。

因此，如果按照当前路径继续推进，很可能被评价为缺乏足够集中、足够有辨识度的算法贡献。

2. 当前选题仍然容易被归入“生命科学助手”子集 已经有另一个同学的毕设是“生命科学助手”，同样采用多 Agent 架构。从架构相似性上来说，即便本系统已经具备一些机制，也很难天然摆脱“生命科学助手系统的一种更为复杂的实现”这一印象。

原因在于，当前系统的主要叙事仍然是：

用户提出生命科学任务 → 多 `Agent` 规划与执行调工具 → 返回结果

只要这个叙事不变，那么无论内部运行机制如何增强，它仍然有较大概率被看作生命科学助手的执行引擎、后端工作流或子模块。这对答辩会有很大风险。

因为该系统很难回答与已有的生命科学助手的本质区别、核心贡献是什么等这些问题。

3. 现有的增强机制更偏向工程健壮性，而非算法创新 当前系统中已经存在许多机制保证实验平台能力，但是从选题的角度去看，它们更多体现的是系统设计的正确性、运行治理等方面，没有进一步上升为统一的决策原理和算法问题。

4. 难度、工作量、风险尚未进入统一决策 此前反复讨论的关键词包括难度、工作量、风险、不确定性、成本和人工介入时机等，这些词都指向一个共同问题：系统缺少的不是更多功能，而是一个能够解释并驱动关键治理动作的统一决策框架。

如果没有这一框架，那么就无法体现为什么某个任务更难、为什么某个任务需要人工介入、为什么某条恢复链路比另一条更合适等问题。

小结 因此，当前选题面临的核心问题可以概括为三点：

第一，系统已经具备较完善的平台能力，但是尚未抽象出一个足够独立的核心算法问题。

第二，只要主叙事仍然停留在“生命科学领域的多 Agent 助手”，就难免被已经存在的父集系统覆盖。

第三，在当前时间约束下，必须尽快把研究对象从“继续扩展系统功能”转向“提炼一个可论证、可验证、可答辩”的核心问题。

也正因如此，后续的研究方向应该不再是“再加入什么能力”，而是究竟把当前系统上升为哪一个更为底层和独立的计算机专业课题的问题。

3 选题目标的重新定义

在前两部分的分析之后，可以看到，当前问题的关键不在于系统是否已经足够复杂，也不在于是否可以继续增加更多功能，而在于现有选题的研究对象需要重新定义。如果这一点不改变，那么后续的所有改进都很可能会继续停留在“把生命科学领域的多 Agent 助手做得更完善”这一层面上，而难以形成一个边界清晰、创新点集中、能独立成立的毕业设计选题。

因此，下一步重要的不是继续扩展原系统，而是转换到研究问题上。

从原有思路看，课题更容易被理解成：构建一个面向生命科学任务（蛋白质设计任务）的多 Agent 系统，通过规划、执行、工具调用和人工确认，帮助完成蛋白质设计任务等科研任务。这种表述的主语仍然是“生命科学助手”。一旦如此，无论其内部加入多少与运行机制，都仍然可能被视为一个更大助手系统中的一部分。也就是说，问题不在于这些机制本身是否重要，而在于它们没有被提升为一个更底层的研究对象。

要解决这一点，课题目标必须从“做一个生命科学助手”转向“研究高代价科学工作流如何被控制”。这一变化在明确毕设的边界。一旦采用新的表述，系统的核心问题就不再是“如何理解生命科学任务并给出结果”，而变成了“在一个代价高、链路长、存在失败与恢复、且允许人工介入的科学工作流中，系统应该依据什么原则决定下一步如何推进”。这样，系统关注的重点也会发生变化：难度、风险、恢复空间、人工介入时机等因素，不再是辅助性的描述目标，而是会成为驱动执行策略变化的核心决策变量。课题的重点就从一个助手或系统，转变成运行时控制与治理机制。

我认为这一重新定义是重要的，因为它同时解决了三个问题：

首先，它能够把课题从一个应用层面的生命科学助手，提升为一个更加通用的计算机问题。只要研究对象被定义成“高代价科学工作流控制”，那么蛋白质设计就不再是课题本体，而只是一个验证场景。这样做的意义在于，系统的核心方法理论上不仅可以服务于蛋白质设计，也可以扩展到其他科学工作流场景，例如复杂文献调研、材料设计或多阶段实验分析。也就是说，课题不再被一个具体应用锁死，而开始具备“方法先于场景”的结构。

其次，这种重新定义有助于切断与“生命科学助手”父集之间的包含关系。如果别人的系统关注的是“如何面向生命科学系统提供智能帮助”，而当前课题关注的是“如

何控制高代价工作流的执行、恢复与人工介入”，那么两者的层级就已经不同。前者关注任务入口和任务处理，后者关注治理逻辑。此时，生命科学助手可以被视为该课题的上层入口，而不是父集。这种关系转换对于选题的独立性尤其重要。

再次，从计算机专业毕设的角度去看，这种重新定义也容易形成明确的算法主线。一旦将命题定义为“工作流控制”，就必须回答一系列典型的计算机问题：系统状态如何表示，哪些变量是不可直接观测的，运行时观测如何更新系统判断，动作空间如何定义，目标函数如何刻画成本、风险和收益之间的平衡，以及在有限预算和不确定性条件下系统如何做出决策。这些问题天然带有控制、调度、约束优化和序贯决策的特征，明显比“继续扩展生命科学助手能力”更接近一个可论证的研究方向。

在这一基础上，新的目标可以概括为：**不再把毕设定义为生命科学多 Agent 助手，而是定义为面向高代价工作流的运行时治理与控制问题，并以蛋白质设计场景为验证对象**。这一目标意味着，现有系统的角色也在随之变化。它不再是毕设本身的全部内容，而是一个实验平台和运行载体。那么，该系统中采用的 FSM、HITL、patch/replan、候选与实验框架等，都不再是零散功能，而是成为后续控制算法可依附的基础环境。换句话说，真正的研究主题转移到该系统之上的运行时控制层。

另外比较重要的是，剩余的周期已经不足两个月。此时及时把目标收缩到“高代价科学工作流如何控制”这一更为聚焦的问题上，那么后续工作将更容易围绕一个明确的主线展开：系统状态如何抽象，难度和风险如何进入决策闭环，控制器如何决定继续执行、恢复、人工介入或终止，以及这种控制相对静态流程究竟能带来什么收益。这样一来，剩余时间更有可能形成一个完整且可答辩的结果。

小结 因此，当前毕设需要一次本质上的选题重构，即从“生命科学多 Agent 系统”转向“面向高代价科学工作流的运行时控制问题”。蛋白质设计仍是其重要的应用场景，但它不再是课题定义本身。只有完成这一层转换，后续关于创新点与可行性的讨论才具有意义。也只有在这一前提下，该课题才可能摆脱被父集系统覆盖的风险，成为一个边界清晰、问题集中、具备算法含量的毕设。

4 候选方向比较

在完成选题的重新定义之后，接下来的问题是在可选的研究方向中，哪一条最适合在当前时间、已有系统基础和答辩要求下收敛成一个成立的毕设课题。从目前的讨论来看，真正有代表性的方向大致可以分成四类：继续沿蛋白质设计系统增强路线推进、转向自动 benchmark、转向工作流编译/验证/修复，以及转向运行时 controller。它们分别对应不同的研究对象、不同的创新重心，也对应不同的风险。

4.1 延续“蛋白质设计系统增强”路线

这是与当前系统最接近的一条路线。其基本思路是继续围绕生命科学任务展开，将系统定义为一个面向蛋白质设计或更广义生命科学任务的多 Agent 助手，并再次基础

上不断补充能力，例如更完整的工作链、更复杂的任务拆解、更细致的门控、更丰富的 HITL 交互，以及更完善的恢复逻辑。

这一条路线的核心是：把系统当作应用产品或综合平台来完善。其关注重点在于系统覆盖了多少能力、能处理多少真实请求、交互链路是否顺畅、执行流程是否稳定。它的优势十分明显：与现有系统最贴近，实现成本最低，也最容易展示出成果。

但它的问题同样明显。首先，这条线路最容易被看成是另一个生命科学助手的自然扩展。因为只要系统的对外叙事仍然是“面向生命科学任务（蛋白质设计）的多 Agent 帮助系统”，那么内部无论加入多少状态机、恢复机制和人工确认逻辑，本质上都没有脱离“助手系统”这一概念。其次，这一条路径的创新点很难集中，往往会分散在多个工程增强点上，例如增加某一类工具、改进某一段流程或补充某种门控逻辑。这些内容单独来看都合理，但是很难压缩成一个明确的算法主线。最后，在当前时间压力下，继续走“大系统增强”的路线很容易导致内容越做越多，主线却越来越模糊。

因此，虽然这条路线工程上最平滑，但是从毕设选题的角度来看，它恰恰是风险最高的一条，因为它最容易落入无法解释核心研究问题是什么的局面。

4.2 自动 Benchmark 路线

自动 benchmark 路线的思路是把研究对象从“如何更好地完成任务”转向“如何系统地评测复杂科学工作流任务”。在这一方向上，系统不再首先关注“输出什么结果”，而是关注：如何构造具有不同复杂度和难度层级的任务，如何让这些任务可复现、可比较、可自动评分，以及如何让 benchmark 真正反映多 Agent 系统在高代价科学工作任务中的能力差异。

从近两年的研究趋势来看，这条路线是成立的，而且学术价值很明确。PaperBench(Starace et al., 2025) 强调的是研究复现实验的细粒度自动评分与 rubric 拆解，BrowseComp(Wei et al., 2025) 强调的是高难度信息寻求任务中的持久搜索，CoLLAB(Mahmud et al., 2025) 则突出协作复杂度和多 Agent benchmark 的可扩展构造。

这一线路的本质是：把“难度”本身变成研究对象。它不像助手系统那样关注“给出什么答案”，而是关注“什么样的任务更难、为什么难、如何被自动构造出来、又如何被可靠评分。”从独立性上来说，这条线路非常安全，因为 benchmark 不是生命科学助手的子模块，反而可以把不同助手都作为 benchmark 的被评测对象。从答辩角度看，也更容易回答“贡献到底是什么”，因为 benchmark 一旦设计得好，本身就是独立研究资产。

不过，这条线路本身也有自己的问题。首先，它的重点不是控制算法，而是在任务生成、难度分层和自动评分机制上。如果自动生成和自动评分不够强，benchmark 很容易退化成“整理了一批测试集”。其次，这条线路虽然能很好地体现“难度”，但是也相对弱化了与当前系统直接结合的优势。也就是说，它更独立，但是与现有系统的耦合不够。最后，从剩余时间来看，如果要同时做好任务构造、难度校准和轨迹级评分，工作量其实不小，切容易在“设计评测体系”这一块耗费过多时间。

因此，自动 benchmark 是一条边界很清晰、不容易成为子集的路线，但是它更像

是“独立评测平台”选题，和当前系统的天然结合度略低。

4.3 工作流编译、验证与修复路线

另一条可能的方向，是把研究结果定义为“从科研目标到可执行工作流的编译、验证与修复”。在这条路线下，重点不是任务本身，也不是最终结果，而是：如何从自然语言目标生成结构化工作流；如何检查工具链的 I/O 兼容性、参数合法性和资源约束；以及如何在执行失败后进行最小修复和后缀重规划。

从方法论上来看，这条路线非常有计算机专业特色，因为它天然带有 `compiler`、`verification`、`schema caching`、`plan repair` 等色彩。现有多 `Agent` 系统设计中，越来越多工作开始强调结构化任务表示、明确的状态机组织以及对工具链和交互协议的形式约束。例如 MetaAgent(Y. Zhang et al., 2025) 这类工作已经开始从显式状态机组织的角度构造多 Agent 系统，而工作流系统本身也越来越强调 plan schema、tool contract 和可验证执行过程。

这条线路的核心是：把科学任务的执行链看作一种可编译、可验证、可修复的程序结构。如果从理论表达上看，它很有吸引力。

但是它在当前语境下有两个显示问题。第一，与现有系统已有能力重合较多。当前系统已经具有显式 FSM、计划候选、patch/replan、候选验证和恢复机制，因此如果再把“工作流编译与修复”作为核心方向，很容易被看成对现有 `planner` 的进一步完善，而不是一个研究主线。第二，这条路线虽然很具有形式化，但是在叙事上较弱。换句话说，它更像“让流程更正确”，而不是“让系统在高代价下做出更好的运行时决策”。从应用意义和系统价值上，可能不够直观。

因此，工作流编译/验证/修复路线是一个理论性较强的候选方向，但是与当前系统的差异化空间并没有想象中大，而且更容易被视为现有 `planner` 和恢复逻辑的深化版本。

4.4 运行时 Controller 路线

`controller` 路线的出发点是把研究对象上升为：在高代价、长链路、带恢复与人工介入的科学工作流中，系统究竟如何决定下一步如何推进。在这一方向上，系统不再被理解成一个“完成生命科学任务的助手”，而被理解成一个“治理工作执行的运行时控制器”。

从近两年的研究来看，这条方向虽然还没有完全同构的现成答案，但是已经有多条重要脉络构成了它的理论背景。在不稳定性建模方面，不确定性思维（Uncertainty of Thoughts，UoT）说明不确定性可以进入 `Agent` 的规划闭环 (Hu et al., 2024)；在推理资源方面，BEST-Route 方法提出依据任务难度与质量阈值，对推理计算资源进行动态调度，实现性能与成本之间的权衡 (Ding et al., 2025)；在推理过程优化方面，QLASS 方法通过对中间决策状态进行价值估计，引导 Agent 在推理空间中的搜索路径，提高推理效率与结果质量 (Lin et al., 2025)；在行动与工具使用层面，EcoAct 从经济性约束出发，对 Agent 的动作选择与工具调用策略进行建模，强调资源受限环境下的最优决策问

题 (S. Zhang et al., 2024)；在多 Agent 系统层面，Why Do Multi-Agent LLM Systems Fail? 与 Which Agent Causes Task Failures and When? 的研究表明，系统失败并不完全来源于单一模型能力的不足，而与协作机制、验证流程、责任归因以及系统治理结构密切相关 (Cemri et al., 2025; S. Zhang et al., 2025)。

这一路线本质是：把“难度、风险、工作量和人工介入价值”统一纳入到一个运行时的决策问题。换句话说，`controller` 关注的不是任务内容本身，而是系统治理动作本身。它所决定的不是“下一句怎么回答”，而是诸如：

- 现在是否值得继续执行；
- 是否应该扩大候选探索
- 是否应切换到保守路径；
- 当前失败更适合 `patch` 还是 `replan`；
- 是否应请求 `HITL`
- 是否应在进一步浪费成本前提前终止

这使得 `controller` 方向同时具有三方面优势。第一，它与当前系统切合度最高。因为现有系统已经有状态机、HITL、patch/replan、阶段化执行链和实验框架，`controller` 可以自然地嵌入这些现有能力之上，而不需要推翻架构。第二，它最容易把“难度”从口头概念变成真正的算法变量。难度不再指示任务描述中的印象，而是需要在执行中动态估计、并据此改变策略的核心状态。第三，它在应用意义上最容易成立。蛋白质设计这类高代价场景天然就会遇到“什么时候继续算、什么时候该求助人类、什么时候必须止损”的问题，因此 `controller` 的价值不仅是理论上的，也非常符合真实科研流程的痛点。

当然，`controller` 方向也不是没有风险。它最大的风险在于，如果最后只做成“几个特征 + 一堆人工阈值 + 超阈值就进入 HITL”的规则系统，那么它虽然还能运行，但创新性会明显不足。因此这条路线真正成立的前提是：`controlle` 必须被明确建模为一个部分可观测、带风险与预算约束的序贯决策问题，而不是一个更复杂的门控模块。

4.5 不同方向的比较与理由

综合来看，这几个方向的差异可以概括为：生命科学助手最贴近现有系统，但独立性最弱，也最容易被已有父集覆盖；自动 benchmark 路线独立性最强，难度问题也最自然，但与现有系统直接结合较弱，且任务生成与自动评分部分在剩余时间内工作量也不小；工作流编译/验证/修复路线理论上较“计算机化”，但与现有 `planner` 的功能边界较近，容易被视为现有平台能力的深化；而 `controller` 路线则位于一个相对平衡的位置，它既能明显提升选题层次，使课题从“助手系统”转向“工作流治理”，又能最大限度复用现有系统作为实验平台，同时还能把“难度、风险、工作量、恢复和 HITL”这些此前分散的概念统一到一个核心决策问题中。

因此，如果在当前时间节点下选择一条既能体现算法性，又能形成明确主线、同时还能落到现有系统的路径，那么 controller 是最值得优先考虑的方向。而在 controller 内部，我准备进一步倾向于信念状态（Belief State)controller，因为它最适合把“难度”定义为一个运行时的隐变量，而不是一个静态标签或单纯的打分结果。

5 Belief-State Controller 作为研究主线的理论依据与问题定义

5.1 从当前系统到控制问题的提出

在前文的分析中，已经可以看到当前系统并不缺少工作流执行机制。无论是显式有限自动机、阶段化蛋白质设计流程、还是 `HITL`、`patch/replan` 与事件审计，这些能力都表明系统已经具备较完整的多 Agent runtime 结构。换言之，当前系统所面临的主要矛盾，已经不再是“能不能把系统跑起来”，而是“在一个高代价、长链路、可失败、可恢复的科学工作流重，系统究竟依据什么原则决定下一步如何执行”。

这个问题之所以重要，是因为蛋白质设计这样的科学工作流并不是简单的问答或单步工具调用任务。其执行过程通常包含多个强耦合阶段，不同步骤的成本和失败代价显著不对称，而且错误往往会沿工作流向后传递。一个系统即便具备了候选生成、结构预测、质量门控和人工确认能力，如果它仍然无法回答“现在是否应该继续计算、切换路径、请求人工接入或提前终止”，那么它本质上仍然只是一个更复杂的执行平台，而不是一个具备独立算法主线的研究系统。因此，在当前系统基础上，最自然的提升方向不是继续扩展设计能力，而是把研究对象提升为工作流运行时的治理控制问题。

也正是在这一层意义上，`controller` 才成为比“继续做生命科学助手”更合适的研究主线。它使系统的核心问题从“完成一个生命科学任务”转变为“如何在风险、成本、恢复空间和人工预算约束下控制科学工作流的推进方式”。这不仅更接近控制、调度和序贯决策问题，也更有助于切断与“生命科学助手”父集之间的包含关系，因为 `controller` 所研究的是更底层的底层治理机制，而不是具体的知识问答或任务编排界面。

5.2 为什么选择 Belief-State Controller

在 `controller` 的不同建模方式中，之所以特别考虑 `Belief-State Controller`，是因为当前系统最关键、也最难被直接观测的因素，恰恰是都具有“隐状态”的特征。系统无法在执行之前明确知道某个任务到底有多难、当前候选是否已经足够可靠、此时的局部失败究竟是因为噪声还是结构性困难、剩余恢复空间是否足够支持继续自动执行、以及当前请求人工确认是否真正具有边际价值。换言之，系统面临的不是一个完全可观测的决策环境，而是一个必须根据运行时证据不断更新判断的不完全信息环境。

如果采用静态评分或规则门控的方式，这些问题通常会被简化为一组人工权重和阈值，例如“如果风险分高于某一阈值则请求 HITL，否则继续执行”。这种做法虽然实

现上简单，但会把“难度”退化为一个固定标签，把“控制”退化为启发式分支选择。这样形成的系统即使在工程上可用，也很难支撑一个有足够算法深度的毕业设计。相反，Belief-State 的价值在于，它允许系统把“难度、风险、证据充分性、恢复余量和人工介入价值”统一看作运行时需要持续估计的 latent state，并通过不断接收到的新观测对其进行更新。这样一来，难度不再只是一个离线打分结果，而成为一个真正进入决策闭环的状态变量。

更重要的是，Belief-State 的引入，使控制问题具备了一个更清晰的算法表达。系统不再只是简单地根据当前观测做局部反应，而是维护关于当前执行状态 belief，并再次基础上决定治理动作。这些治理动作不是普通的工具选择，而是工作流层面的决策，例如是否继续执行、是否扩大探索深度、是否切换到保守路径、是否进入局部 patch、是否升级为 suffix replan、是否请求人工确认、以及是否提前终止。也就是说，Belief-State Controller 的核心不在于替代 Planner 或 Executor，而在于作为它们之上的运行时决策层，对工作流的推进方式进行统一治理。这一点与当前系统已经具备的 FSM、HITL 和恢复链机制高度契合，也意味着 Belief-State Controller 既能利用现有系统作为实验平台，又不会退化为现有模块的简单重复。

5.3 本文采用的 Belief-State Controller 问题定义

在本课题中，Belief-State Controller 所面对的对象不是单轮对话，也不是单次模型推理，而是一条高代价科学工作流。该工作流可以被抽象为一个带有数据依赖与控制依赖的图结构，其中每个节点对应某一阶段的任务执行、质量判断、恢复操作或人工确认点。对于蛋白质设计场景而言，这样的图至少包含候选生成、结构预测、质量门控、结构条件优化以及恢复等关键部分。

在任一时刻，系统都处在一个无法被完全观测到运行状态之中。这个隐状态并不是单一的 difficulty score，而是一个结构化的状态集合，用于共同刻画当前任务的真实执行难度、当前证据是否充分、当前工具链是否稳定个、当前恢复空间是否仍然充足、以及当前是否值得引入人工干预。系统所能获得的只是运行过程中的局部观测。例如候选结果的一致性和分歧程度、结构预测输出及其质量标准、质量门控的拒绝原因、最近一次 patch 或 replan 的结果、已经消耗的时间与资源，以及此前人工决策所留下的轨迹。controller 的任务，正是依据这些异构运行时观测，持续更新系统对于当前隐状态的 belief，并据此在若干治理动作之间做出选择。

由此，本文中的 Belief-State Controller 可以被理解为一个面向科学工作流的部分可观测治理控制器。它不直接生成最终任务结果，而是以工作流的运行方式为控制对象，在成本、风险、时间和人工预算约束下，决定工作流下一步应如何推进、恢复、暂停或终止。这个定义具有两层意义：一方面，它使“难度”和“风险”真正进入系统决策闭环；另一方面，它也使课题从一个面向生命科学的助手系统，转化为一个面向高代价科学工作流的控制问题。

5.4 相关研究现状及其启示

围绕这一部分，现有研究已经提供了若干重要启示，但尚未直接给出与本课题完全重合的解决方案。

第一类相关工作是不确定性感知的 agent 规划。例如 UoT(Hu et al., 2024) 提出在信息不足时让 `language agent` 根据不确定性主动进行信息搜索，这说明不确定性感知规划（uncertainty-aware planning）本身已经是被认可的研究方向，也证明“不确定性不是只能被动承受，而可以进入决策闭环”这一思路是成立的。与本课题相比，这类工作更多关注信息寻求与提问行为，而较少涉及高代价工作流中的阶段化恢复、人工介入和治理动作，因此为本研究提供的是方法论启发，而不是现成答案。

第二类相关工作是成本感知路由与 `test-time` 资源控制。BEST-Route(Ding et al., 2025) 将问题难度（query-difficulty）与质量阈值纳入模型路由和采样预算分配，EcoAct(S. Zhang et al., 2024) 则从经济性角度讨论 `agent` 系统中何时注册何种动作或工具。它们共同说明，在大模型系统中，根据难度和成本动态分配计算资源已经成为清晰的研究问题。这与本课题高度相关，因为 `Belief-State Controller` 同样需要根据状态估计决定是否继续投入更多执行决策。但不同之处在于，这些工作通常停留在模型路由或工具使用层面，而本文所关注的，是工作流层面的恢复、人工确认和执行治理。

第三类相关工作是长程价值引导与搜索控制。例如 QLASS(Lin et al., 2025) 通过逐步价值（stepwise value）引导 agent 在复杂任务中做更好的中间选择，Language Agent Tree Search(Zhou et al., 2024) 则通过树搜索统一推理、行动与规划。这些工作表明，大模型 agent 的关键不只在于最终输出，更在于中间决策质量以及对未来收益的估计能力。对本课题而言，其重要意义在于说明：在长链路任务中，控制“下一步怎么走”本身就是研究重点。然而，这类任务仍更偏向搜索效率和推理路径优化，而不是面向高代价工作流的治理控制。

第四类相关工作是多 Agent 系统失败分析与结构化设计。Why Do Multi-Agent LLM Systems Fail?(Cemri et al., 2025) 指出，多 Agent 系统的失败很大一部分并非来自模型本身，而来自角色失配、接口不清以及验证不足和终止条件设计不良。Which Agent Causes Task Failures and When?(S. Zhang et al., 2025) 进一步说明，多 Agent 链式执行中的失败归因并不容易。于此同时，MetaAgent(Y. Zhang et al., 2025) 这类工作则显示，有限状态机等显式结构已经开始被用于更系统化地组织多 Agent 系统。这些研究共同提供了重要背景：在多 Agent 系统中，“如何治理执行过程”已经成为一个独立问题，而不是简单附属于任务求解过程。对本课题而言，这些工作一方面说明选择 `controller` 方向是合理的，另一方面也表明真正的难点不在于“增加一个模块”，而在于如何在带协作噪声和恢复链的环境中稳定做出治理决策。

综合这些文献可以看出，现有研究分别从不确定性、成本约束、价值引导、失败分析和结构化设计等角度接近了本课题，但尚未把以下几个因素统一起来：高代价科学工作流、显式恢复动作、HITL 动作、结构化工作流状态，以及运行时 `belief` 更新。也正因如此，`Belief-State Controller` 仍然具有明确的研究空间。它既不是凭空提出一个完全脱离文献背景的概念，也不是简单复用已有方法，而是在现有研究基础上，将问

题收束到一个更贴近当前系统、也更贴近高代价科学工作流本质的问题上。

5.5 为什么是 Belief-State Controller

因此，将当前选题进一步收束为 Belief-State Controller，主要是因为它同时满足了三个条件。首先，它最能回应当前系统的真正缺口，即运行时控制依据的缺失。其次，它最能把“难度、风险、工作量、恢复与人工介入”统一为一个明确的算法问题，而不是若干零散机制的堆叠。最后，它在现有研究中有清晰的相关脉络可依托，又没有被已有工作完全覆盖，从而具备理论依据，也保留了可用于毕业设计的创新空间。

从系统角度看，当前已有的 FSM、HITL、patch/replan 和实验框架提供了 controller 的运行环境；从研究角度看，Belief-State 为系统如何理解和更新“当前究竟处于什么状态”提供了更有力的算法表达。也正因如此，Belief-State Controller 不只是一个可行方法，而是当前条件下最有可能同时兼顾算法性、独立性、系统可落地与时间可行性的方向。

6 可能的创新点与难点

在前文已经明确 Belief-State Controlle 的问题定义之后，接下来的问题便是：如果将其作为毕业设计的核心方向，究竟有哪些内容可以被合理地视为创新点，又有哪些问题会构成该方向真正的研究难点。只有把这两个问题分清楚，课题才能避免陷入两个误区：一方面，把已有概念包装成创新；另一方面，低估工作流控制问题在算法建模上的复杂性。

6.1 创新点首先不应在使用 Belief State 本身

首先必须明确，Belief State 这一概念本身并不能构成创新点。无论是在经典部分可观测决策问题中，还是近两年的 agent 控制、test-time routing 与不确定性感知规划工作中，围绕隐状态、不确定性评估和基于 belief 的决策思想都已有较充分的研究基础。因此，如果课题只是简单宣称“将 belief-state 引入蛋白质设计多 Agent 系统”，这样的表述很难成立。

真正可能成立的创新，不在于是否使用了一个已有的控制思想，而在于是否针对当前这一特定问题，提出了新的问题形式化、新的状态设计、新的观测更新方式和新的动作组织方式。换句话说，Belief-State 只是方法论背景，而不是贡献本身。课题的创新只能来自于：如何把一个高代价、长链路、带恢复和人工介入的科学工作流，转化为一个此前尚未被充分刻画的控制问题。

6.2 可能的创新点一：新的问题形式化

在当前语境下，最有分量的创新点首先来自问题定义本身。现有研究虽然已经分别处理了不确定性感知规划、成本感知路由、长程搜索价值估计以及多 Agent 失败分析，

但较少将“高代价科学工作流”作为一个统一的治理控制对象进行建模。特别是在当前系统中，工作流不仅有明显的阶段结构，而且包含恢复动作、人工确认点以及代价高度不对称的执行路径。这意味着系统面对的不仅是一个普通的查询路由或工具选择问题，而是一个带有恢复链和治理动作的部分可观测控制问题。

如果能明确提出：高代价科学工作流应被形式化为一种带恢复动作、HITL 动作和预算约束的部分可观测治理控制问题，那么这本身就是一个重要的创新起点。它使课题的研究对象脱离了“生命科学助手”的语境。而进入了更抽象、更通用的工作流控制层。也正因如此，这一形式化不仅服务于当前蛋白质设计场景，也有潜力迁移到其他高成本科研工作流中。

6.3 可能的创新点二：结构化隐状态表示

如果问题的形式化是课题的外壳，那么真正的算法内核则很可能落在隐状态设计上。一个简单的做法，就是给系统定义一个单一的难度分数（difficulty score），并据此决定是否继续执行或请求 HITL。但这种做法往往过于粗糙，它既不足以表现真实运行过程中的多维状态，也很容易退化成一个启发式分数系统。因此，若要体现算法含量，更合理的方式是将该系统的运行状态拆解为一组隐状态。

这一状态表示不应只包括“当前任务难度”，还应同时考虑：当前证据是否充分、当前工具链是否稳定、当前恢复空间是否仍然充足、当前继续执行的边际收益是否大于其潜在代价，以及当前请求人工介入是否真正具有价值。这样依赖，`Belief-State Controller` 所维护的就不再是一个简单的风险标签，而是一种关于未来执行质量、成本和治理收益的联合信念表示。这种状态设计的核心在于，它让控制器不再只是对单一指标做反应，而是对整个运行过程的多维条件做综合判断。

这一部分非常可能成为最核心的算法创新点之一。因为相比直接套用现有控制理论框架，如何设计符合高代价科学工作流特征的状态变量，才是本课题真正具有独特性的地方。现有研究已经证明不确定性和价值评估可以进入 agent 决策，但对于“高代价科学工作流中的难度、恢复余量和人工介入价值如何统一进入状态空间”仍然缺少直接答案。

6.4 可能的创新点三：基于运行时事件的 belief 更新机制

如果说隐状态设计决定了系统“应该关心什么”，那么 belief 更新机制决定的就是系统“如何根据运行时证据改变自己的判断”。这一点在本课题中同样可能构成非常有价值的创新点。

原因在于，当前系统的观测并不是单一类型的反馈，而是高度异构的运行时事件。系统可能观察到候选之间的一致性或分歧，结构预测及其质量指标，质量门控的失败码和拒绝原因，`patch` 和 `replan` 的执行结果，人工确认记录，以及已经消耗的事件和资源。不同观测之间的语义差异极大，且可靠性也不完全相同。如果控制器不能把这些异构的观测有效整合为对“当前系统状态”的更新，那么 `Belief-State` 很快就会沦为一个概念性装饰，而无法真正发挥作用。

因此，另一个较强的潜在创新点便是：提出一种基于运行时异构事件的 belief 更新方式。这个更新方法不一定需要追求完整的概率图模型或最优贝叶斯推断，但至少需要使控制器能够在观察到新的失败、候选分歧、门控结果或人工介入口，对自身的风险判断和动作偏好进行合理修正。对于毕设而言，这一部分的价值在于，它比简单的打分更具动态性，也比静态规则更具算法性；同时，它又能自然地利用当前系统内已有的 EventLog、状态机和恢复链路作为观测来源，因此在理论和系统之间形成较好的连接。

6.5 可能的难点一：难度和风险是隐变量

与上述创新点对应，这一方向也存在若干真实而不可忽略的研究难点。首先，也是最核心的一点就是，controller 真正想知道的那些量——例如当前任务是否已经进入高风险状态、当前失败是否预示结构性的困难、当前继续自动执行是否仍有价值——都不是系统可以直接观测到的。系统看到的只有局部证据，而不是完整的真实状态。因此，如何从这些有限、噪声化、阶段相关的观测中推断出当前系统工作流所处的真实状态，本身就是一个困难问题。

这意味着，课题不能把“Belief-State”写成一个抽象词汇后就结束，而必须面对：究竟哪些变量应该被视为隐状态，哪些只是观测特征，哪些变量虽然想要纳入状态空间，却在当前实验条件下无法稳定获得。若这一层划分不清晰，controller 就很容易回退到规则系统，因为这时 belief 更新集那个只是对若干可观测特征的机械加权，而不是对系统状态的真正估计。

6.6 可能的研究难点二：观测异构且噪声显著

另一个关键难点来自运行时观测本身的异构性与不稳定性。当前系统中的观测信息来源极其多样：候选分布、结构质量、门控拒绝码、失败历史、恢复轨迹、人工确认记录等，它们在语义层次和统计特征上差异很大。某些观测是连续质量信号，某些观测是离散错误码，某些则是具有上下文依赖的历史事件。此外，这些观测并不总是可靠的：结构预测本身可能存在波动，门控结果未必完全反映真实根因，人工确认也未必总具有一致性。

这就导致一个明显难点：即使状态设计是合理的，系统仍然需要面对“不同类型证据如何被统一解释”的问题。对毕业设计而言，这种难点并不要求完全以复杂概率模型解决，但至少需要在方法上给出明确说明：控制器为何相信某类观测更重要，何时应该因为某类观测而调整 belief，何时则应保持保守。也就是说，课题不仅要回答“看到了什么”，还要回答“系统为何因此改变判断”。

6.7 可能的研究难点三：动作代价高度不对称且具有长期影响

相较于普通问答任务，科学工作流的另一个突出特点是动作代价高度不对称。某些动作几乎没有额外成本，例如继续沿当前路径多执行一步；而另一些动作则可能带来明显后果，例如一次无意义的高成本结构预测、一次错误的后缀重规划、一次过早发起或

过晚触发的人工确认。这些动作不仅代价不同，而且还会影响后续恢复空间和执行效率，因此它们的价值不能只看当前收益，而必须放到更长时间范围内理解。

这也是 controller 相较于静态门控更复杂的地方。它不能只判断“当前哪个动作看起来最好”，而必须估计当前动作会如何改变未来的风险、成本和可恢复性。因此，本课题的难点之一在于如何让控制器真正体现出对长期后果的考虑，而不是仅仅根据当前局部状态做贪心反应。如果不能解决这一点，控制器就很容易在局部上显得合理，却在全局上导致更多恢复和更高代价。

6.8 可能的工作难点四：多 Agent 环境会引入额外的协作噪声

最后，还必须认识到本课题的控制环境并不是一个“干净”的单模型决策环境，而是一个由多个角色、多个阶段和多种恢复链共同构成的多 Agent 执行环境。这意味着系统除了面对任务本身的不确定性，还要面对协作层面的问题，例如角色漂移、接口不匹配、错误归因困难以及阶段之间信息传递失真等。近年的多 Agent 文献已经反复表明，这类失败并不少见，而且往往并非由某个单点模块造成，而是由整个协作链条共同诱发。

这会直接影响控制器的设计，因为 controller 所依赖的运行时信号，本身就可能已经受到协作噪声的污染。也就是说，controller 不仅要对任务状态进行推断，还要在一定程度上面对“观测本身可能已经失真”的现实。这使得控制问题比普通单体模型路由或推理控制更复杂，也解释了为什么本课题虽然看似只是“做一个控制器”，实质上却具有相当高的研究难度。

6.9 本章小结

综合来看，Belief-State Controller 方向的创新空间主要不在于“提出了 belief-state 这一概念”，而在于是否能够围绕当前高代价科学工作流场景，完成新的问题形式化、结构化隐状态设计、运行时事件驱动的 belief 更新以及工作流级治理动作组织。与此同时，这一方向的真正难点也不在于编码实现本身，而在于如何从隐变量、异构观测、长期代价和多 Agent 协作噪声中提炼出一个足够清晰、足够有解释力的控制问题。

也正因如此，这一方向既具备较明确的算法创新潜力，又能够体现出足够的研究难度。但需要强调的是，这些创新和难点只有在问题边界始终被严格控制在“高代价科学工作流的运行时治理”这一层时，才会形成真正有效的选题支撑；一旦重新退回到“生命科学助手增强”这一叙事，所有这些算法层面的价值都可能再次被冲淡。

7 可行性与时间评估

在前文已经完成选题重构、方向比较以及 `Belief-State Controller` 的问题定义之后，最后必须回答一个最现实的问题：在当前时间已经接近三月底、距离毕业设计收尾不足两个月的条件下，这一方向是否仍然具有可行性。这个问题不仅关系到技术安

排，更关系到选题是否真正能够收敛为一个可答辩的毕业设计。因此，本节不再讨论该方向“理论上是否有意义”，而是集中分析它在现阶段是否具备现实落地条件，以及在有限时间内应如何控制范围。

7.1 可行性的前提：现有系统已经提供了足够的实验平台

判断当前方向是否可行，首先要看是否仍需从零搭建系统。如果答案是“需要重写一套全新的运行时平台”，那么在现阶段继续转向显然不现实；但如果现有系统已经提供了大部分实验所需的基础，那么控制器方向的可行性就会显著提高。结合前文对当前系统的分析，可以看到后者更接近真实情况。

当前系统已经具备显式有限状态机、阶段化蛋白设计流程、候选生成与质量门控、`patch/replan` 恢复链以及 HITL 机制。这意味着工作流的基本执行逻辑、可恢复执行链路和人工确认链路都已经存在。对于 `Belief-State Controller` 而言，这一点非常关键，因为控制器本身并不需要重新实现整个任务系统，它所依附的正是这些已有的运行时结构。换言之，当前课题不是从零开始做一套 agent 平台，而是在既有平台之上补足一个“如何依据运行时状态做治理决策”的核心算法层。

更重要的是，系统还已经具备实验框架和评测基础。这意味着后续工作不需要再从头建设完整评测体系，而可以在现有实验能力之上，增加控制器与基线策略的对比。就时间成本而言，这一前提极大降低了转向 controller 的风险，因为真正需要新增的重点将不再是大规模平台建设，而是：问题形式化、状态定义、决策逻辑设计以及实验验证。

7.2 为什么在当前时间节点下，Belief-State Controller 仍然可做

虽然距离毕业设计结束时间已经不长，但 `Belief-State Controller` 方向仍然具有可行性，其原因不在于它“足够简单”，而在于它具备一个很重要的结构性优势：它可以建立在现有系统之上，以“新增控制层”的方式展开，而不要求对底层架构进行推翻式改造。

这意味着，课题的工作量可以被控制在一个较聚焦的范围内。系统的任务流程、恢复机制、人工确认逻辑和事件记录能力都已经存在，后续真正需要完成的核心工作，是围绕这些已有能力提出并验证一个统一的控制问题。只要研究范围被严格限制在“高代价科学工作流的运行时治理”，那么课题就不必再继续扩展任务覆盖面，也不必把精力投入到新的工具接入、全新流程建设或大规模产品化工作中。对于当前阶段而言，这种“在已有平台上增加新的研究层”的方式，是比继续做一个更大的助手系统更可行的。

与此同时，`Belief-State Controller` 的可行性还来自其可做“降阶版本”。前文已经讨论过，完整的 POMDP、端到端强化学习或复杂概率图推断虽然理论上更强，但在当前时间条件下风险过高。相比之下，一个结构化的、显式的、可解释的 `Belief-State Controller Lite` 更适合作为毕设收敛方向。它不追求在理论上求得最优控制策略，而是强调：如何定义合适的 latent state，如何根据运行时异构观测更新 belief，以及如何基于更新后的 belief 做工作流级治理动作选择。这样的工作量虽然不轻，但它是可管理的；更重要的是，它仍然能保持明显的算法主线，而不会退化成纯粹的工程堆叠。

7.3 本课题的时间可行性建立在“严格收缩研究边界”之上

需要强调的是，controller 方向之所以可行，并不是因为它本身没有难度，而是因为它可以通过边界控制收敛到一个适合当前阶段完成的版本。若继续让课题同时承担“生命科学助手增强”“多场景扩展”“完整 benchmark 设计”“复杂控制算法求最优策略”等多重目标，那么无论选择哪个方向，都很难在不足两个月内完成。因此，本课题要成立，必须明确哪些内容是本阶段应该做的，哪些内容则应主动放弃。

首先，不应再将“构建一个更完整的生命科学助手”作为目标。该目标过于宽泛，且会不断诱导系统能力扩张，削弱主线。其次，不应把课题范围扩展到多种不同科学场景，而应坚持以蛋白设计工作流作为唯一核心验证场景。再次，不应试图实现完整的 POMDP 求解、端到端强化学习或高度复杂的最优策略学习，而应聚焦于一个结构化、可解释、可实验验证的 belief-state 控制原型。最后，不应将自动 benchmark 也一并做成并列主贡献，而应将现有实验能力仅作为 controller 的验证载体。

也就是说，当前时间可行性的前提，不是“课题足够简单”，而是“课题目标被主动收束”。这种收束一旦做到，课题就会从“大而散的系统建设”转变为“围绕单一控制问题的集中验证”，这正是当前阶段最重要的项目管理条件。

7.4 剩余时间内最合理的工作推进逻辑

从时间安排上看，当前阶段的工作应围绕“先收敛问题，再构造方法，最后做验证”这一顺序展开，而不能反过来先做大量系统改动，再试图从中提炼论文主线。就现实情况而言，剩余时间更适合被划分为三个连续阶段。

第一阶段是问题收敛与方法定型阶段。该阶段的核心不是编码，而是进一步压实研究对象，明确控制器所维护的状态、可用的观测、动作空间以及目标函数，并将 `Belief-State Controller` 与静态门控、规则系统或固定策略清晰区分开来。这一阶段决定课题是否有资格被称为“算法问题”，也是后续所有实验是否有意义的前提。

第二阶段是控制器原型与基线对比阶段。在问题定义明确之后，需要基于现有系统形成一个可运行的控制原型，并设定至少一类静态基线作为对照。这里的关键不在于功能做得多复杂，而在于让控制器真正改变工作流治理动作，并能够在实验中体现出与固定策略的差异。只有当控制器能在“继续执行、恢复、人工介入、终止”等关键节点上做出不同于现有规则的选择时，它才真正构成研究对象。

第三阶段是实验分析与论文收尾阶段。由于当前系统已有实验基础，这一阶段应重点围绕 controller 的收益和局限性展开，包括成功率、恢复代价、人工介入次数、执行时间以及失败样本分析等。对于毕业设计而言，这一阶段的重要性并不低于算法本身，因为导师与答辩老师最终看到的不是单个模块是否“能跑”，而是这一方向是否在合理的比较条件下体现出价值。

7.5 为什么这一时间安排仍然具有现实性

上述推进逻辑之所以在当前时间下仍然现实，是因为它建立在“新增算法层，而非重写系统”的基础之上。就工作量结构而言，后续最重的部分并不是工程开发，而是研究问题的明确表达与实验验证逻辑的搭建。这一点对毕业设计非常重要，因为在有限时间内，真正有价值的不是继续积累代码规模，而是尽快让课题形成一个清晰的论证闭环。

从这个角度看，当前方向最大的优势就在于：现有系统已经承担了大量底层工程工作，使得剩余阶段可以把重心转移到控制层设计与论证上。只要不再继续扩大系统边界，这种工作安排仍然是现实的。换句话说，controller 方向不是因为“实现很轻”而可行，而是因为“现有平台已经足够重”，从而使新增部分可以被压缩在一个可管理的算法层内。

7.6 当前方向的主要时间风险

尽管如此，`Belief-State Controller` 方向仍存在明确的时间风险，主要集中在三个方面。

第一，问题定义如果迟迟不能收敛，后续实现和实验就会持续摇摆。由于 controller 的边界天然比普通模块更抽象，一旦在状态、动作或目标函数上反复修改，时间会迅速被消耗在概念调整上，而非形成有效结果。第二，如果在实现时试图同时追求过强的方法复杂度，例如完整的 POMDP、复杂学习式策略或多场景泛化验证，则工作量会显著失控，最终既难以完成，也难以保证结果质量。第三，如果实验目标设定过多，试图同时证明所有维度上的提升，那么有限时间内很难形成有说服力的核心结论。

因此，时间风险并不意味着该方向不可行，而是意味着必须坚持“单主线、单场景、单核心问题”的收敛策略。只要这一点被严格执行，本课题仍有较大概率在剩余时间内形成完整成果；反之，如果重新回到多目标并行推进的状态，那么任何方向都会迅速变得不可控。

7.7 本节结论

综合来看，`Belief-State Controller` 方向在当前时间节点下仍然具备现实可行性，其根本原因在于：现有系统已经提供了足够完整的运行时平台和实验基础，使得后续工作可以集中在控制层的研究问题上，而无需从零搭建新的系统框架。同时，这一方向又允许通过“降阶版本”控制方法复杂度，从而避免陷入完整 POMDP 或强化学习等高风险路线。

但这种可行性并不是无条件的。它建立在三个前提之上：其一，课题必须从“生命科学助手增强”收缩为“高代价科学工作流治理控制”；其二，验证场景必须收敛到当前已有的蛋白设计工作流；其三，方法目标必须控制在一个可解释、可验证的 `Belief-State Controller Lite` 范围内。只有在这三个条件同时满足时，controller 才不仅在理论上

值得做，而且在当前不足两个月的时间条件下依然是可以完成、可以论证、也可以答辩的。

8 风险与备选方案

References

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent llm systems fail? https://arxiv.org/abs/2503.13657

Ding, D., Mallick, A., Zhang, S., Wang, C., Madrigal, D., Garcia, M. D. C. H., Xia, M., Lakshmanan, L. V. S., Wu, Q., & Rühle, V. (2025). BEST-route: Adaptive LLM routing with test-time optimal compute. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=tFBIbCVXkG

Hu, Z., Liu, C., Feng, X., Zhao, Y., Ng, S.-K., Luu, A. T., He, J., Koh, P. W., & Hooi, B. (2024). Uncertainty of thoughts: Uncertainty-aware planning enhances information seeking in large language models. https://arxiv.org/abs/2402.03271

Lin, Z., Tang, Y., Yao, X., Yin, D., Hu, Z., Sun, Y., & Chang, K.-W. (2025). QLASS: Boosting language agent inference via q-guided stepwise search. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=f6lio2CZIM

Mahmud, S., Bagdasarian, E., & Zilberstein, S. (2025). CoLLAB: A framework for designing scalable benchmarks for agentic LLMs. *Workshop on Scaling Environments for Agents*. https://openreview.net/forum?id=372FjQy1cF

Starace, G., Jaffe, O., Sherburn, D., Aung, J., Chan, J. S., Maksin, L., Dias, R., Mays, E., Kinsella, B., Thompson, W., Heidecke, J., Glaese, A., & Patwardhan, T. (2025). Paperbench: Evaluating ai's ability to replicate ai research. https://arxiv.org/abs/2504.01848

Wei, J., Sun, Z., Papay, S., McKinney, S., Han, J., Fulford, I., Chung, H. W., Passos, A. T., Fedus, W., & Glaese, A. (2025). *Browsecomp: A simple yet challenging benchmark for browsing agents*. https://arxiv.org/abs/2504.12516

Zhang, S., Yin, M., Zhang, J., Liu, J., Han, Z., Zhang, J., Li, B., Wang, C., Wang, H., Chen, Y., & Wu, Q. (2025). Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=GazlTYxZss

Zhang, S., Zhang, J., Ding, D., Garcia, M. H., Mallick, A., Madrigal, D., Xia, M., Rühle, V., Wu, Q., & Wang, C. (2024). Ecoact: Economic agent determines when to register what action. https://arxiv.org/abs/2411.01643

Zhang, Y., Liu, X., & Xiao, C. (2025). Metaagent: Automatically constructing multi-agent systems based on finite state machines. *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=vOxaD3hhPt

Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., & Wang, Y.-X. (2024). Language agent tree search unifies reasoning, acting, and planning in language models. *Forty-first International Conference on Machine Learning*. https://openreview.net/forum?id=njwv9BsGHF